

Operating Systems (OS)

Structure of Operating System

Prof. Dr.-Ing. Jörg Abke



TH Aschaffenburg
university of applied sciences

Copyright Hinweise

© Jörg Abke

Dieses Werk ist urheberrechtlich geschützt. Alle Rechte, insbesondere Vervielfältigung, Verbreitung oder Übersetzung liegen beim Autor. Kein Teil des Werkes darf ohne Genehmigung des Autors, auch nicht für Unterrichtszwecke, fotokopiert oder durch andere Verfahren reproduziert oder verbreitet werden. Ebenfalls ist die Einspeicherung in EDV-Anlagen ohne Genehmigung des Autors nicht gestattet. Jeder Studierende, der an der entsprechenden Lehrveranstaltung des Autors an der Technischen Hochschule Aschaffenburg teilnimmt, ist berechtigt, einen Ausdruck dieses Werkes anzufertigen und für die Zwecke seines Studiums zu verwenden. Eine weitergehende Verwendung bedarf in jedem Falle der ausdrücklichen Zustimmung des Autors. Gebrauchsname, Handelsnamen, Warenbezeichnungen etc. werden in diesem Werk ohne Gewährleistung der freien Verwendbarkeit benutzt. Texte, Abbildungen, Programme und technische Angaben wurden sorgfältig erarbeitet. Trotzdem sind Fehler nicht völlig auszuschließen. Der Autor weist darauf hin, dass er für Fehlerfreiheit keine Gewähr und für eventuelle Folgen keine Haftung übernimmt.

Process Context

- **All resources** regarding **each process**, which **have to be managed** by an OS

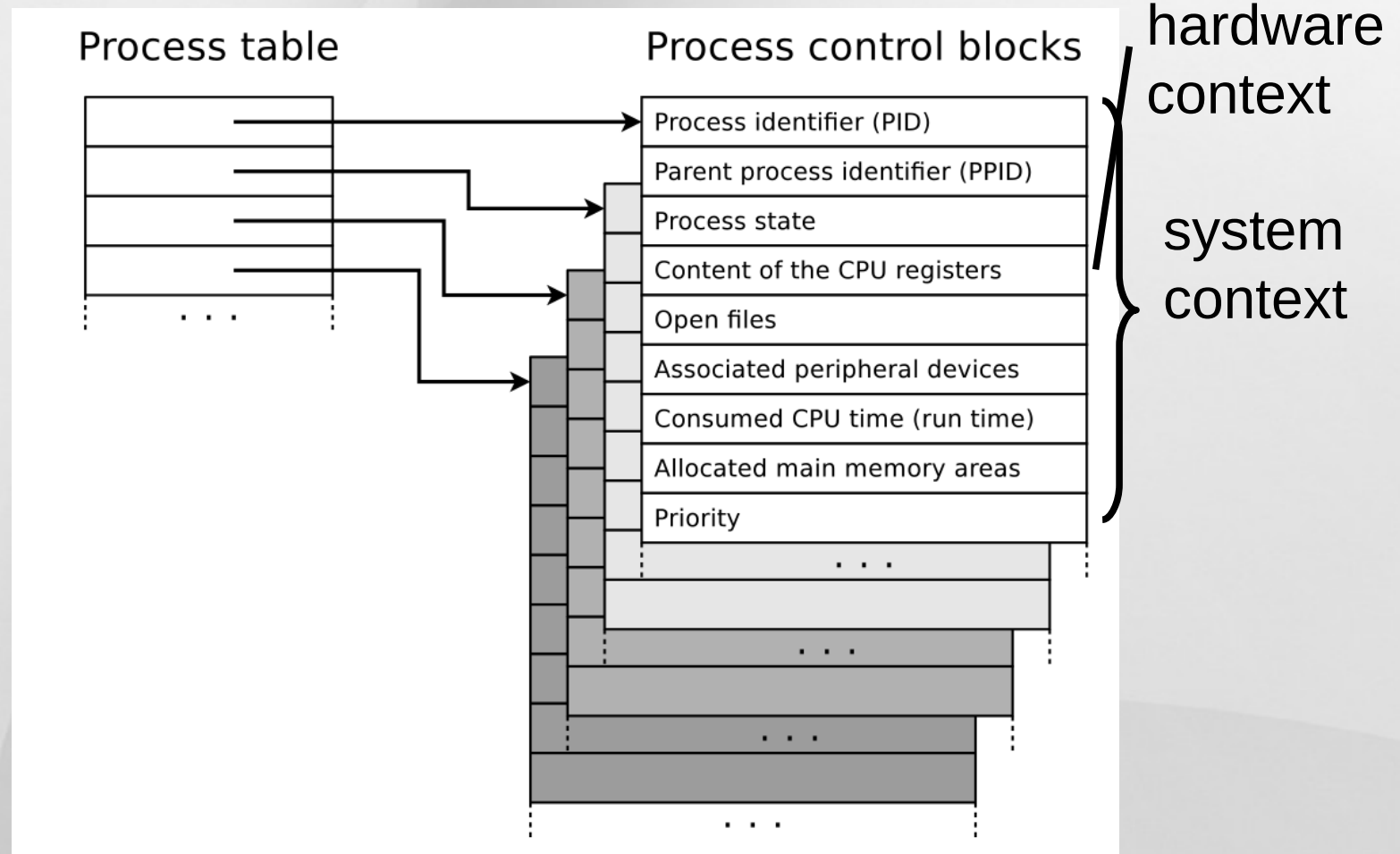
3 Parts

- **Hardware context**
 - Contents of selected CPU registers to get back into current process state
- **System context**
 - Operating System's data regarding process
- **Hardware and System context are kept in the Process Control Block (PCB)**
- **User context**
 - Pages of the allocated virtual memory address space

Process Table and Process Control Blocks (PCBs) – OS perspective

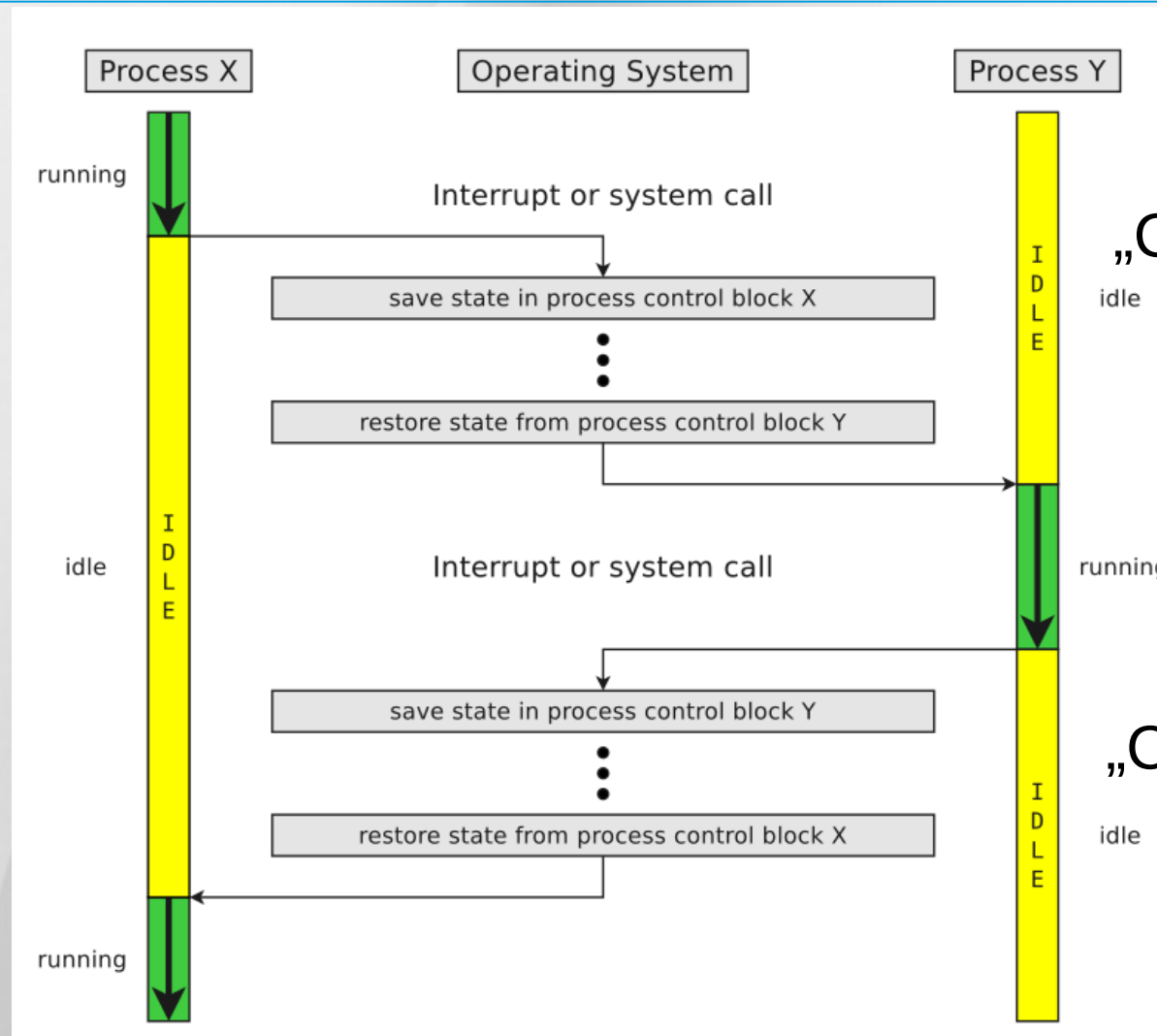


TH Aschaffenburg
university of applied sciences



Source: C. Baun, Operating Systems

Process X is switched to Y

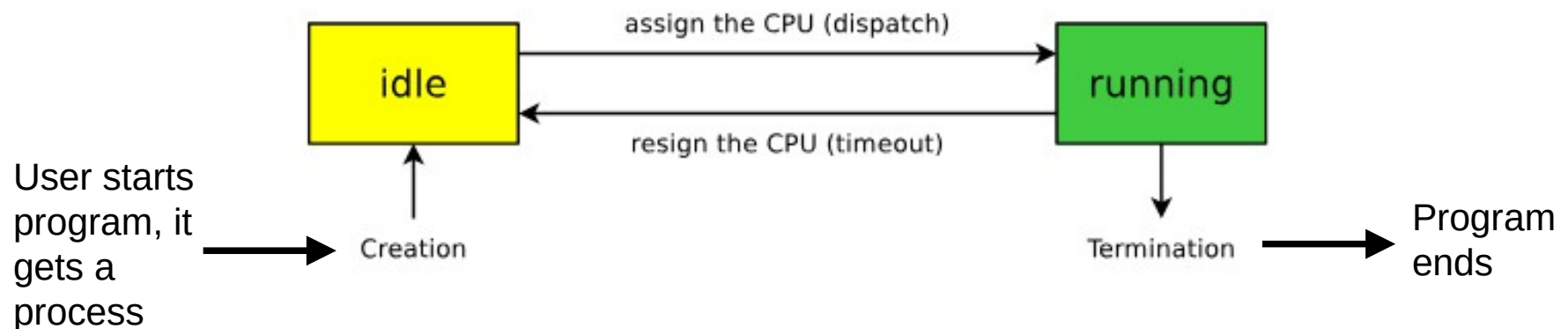


„Context Switch“

„Context Switch“

Process States

- Each **Process** is in a **state** at any point in time
- Process state is essential part of system state
- Number of process states depends on OS's Process State Model
- Simplest 2-State Model



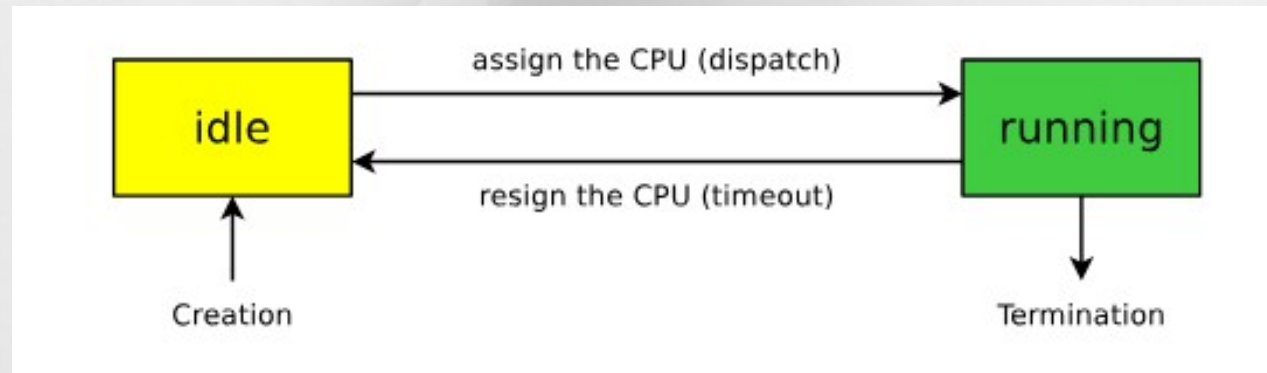
Responsible OS Parts – Dispatcher & Scheduler



TH Aschaffenburg
university of applied sciences

- **Dispatcher**
 - Handles state transition
 - Context switch
- **Scheduler**
 - Controls the dispatcher
 - Decides on process state change (running, resuming) using a scheduling algorithm(s)
 - It controls the execution order of processes
- **What might be the reason for separating Dispatcher from Scheduler**

Pros and Cons of 2-State Model

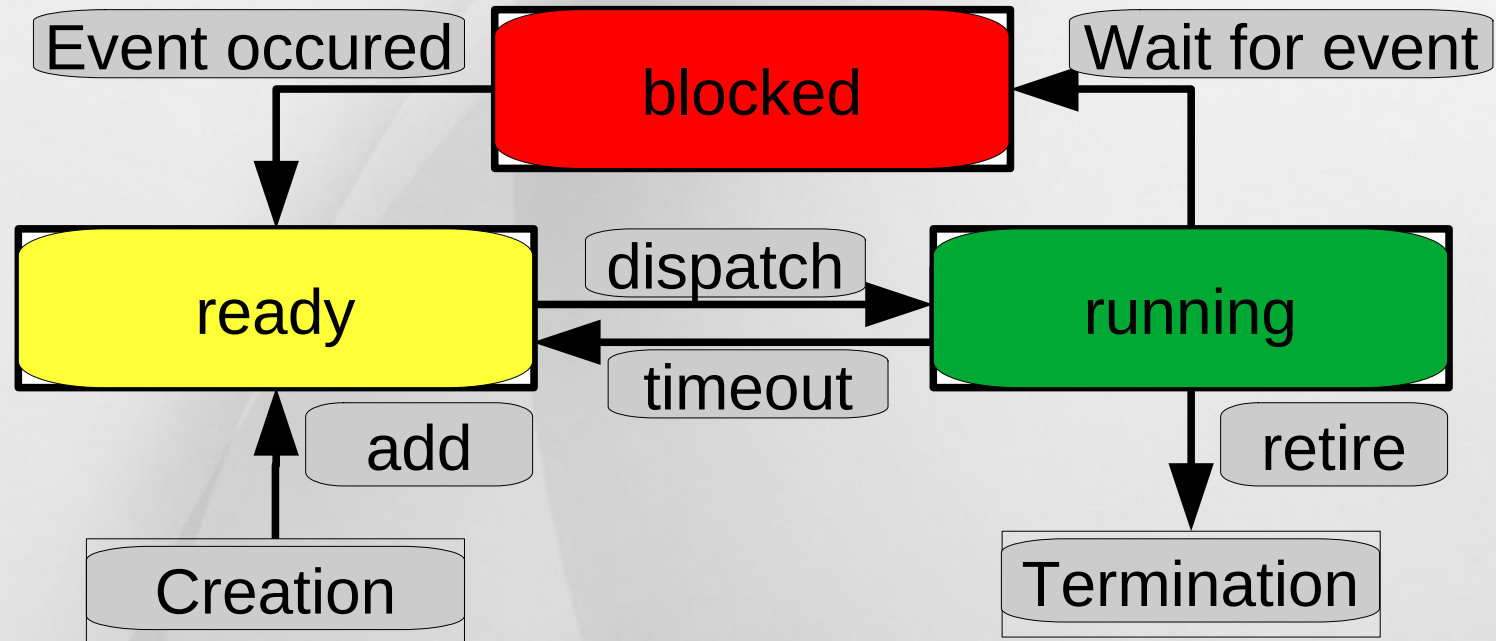


- **Pros: Simple Model**
- **Cons: Assumption is wrong, that any process is ready to run at any time**
 - **Why is this assumption wrong?**

3-State Model



TH Aschaffenburg
university of applied sciences



- **Adding a blocked state to the 2-State Model**
 - Running
 - Idle renamed to Ready
 - Blocked

Expansion from 3 to 5 States

Reasons:

- Limitation of number of ready processes on the system (memory space limitation)
- Intermediate state before loading program from mass storage into memory

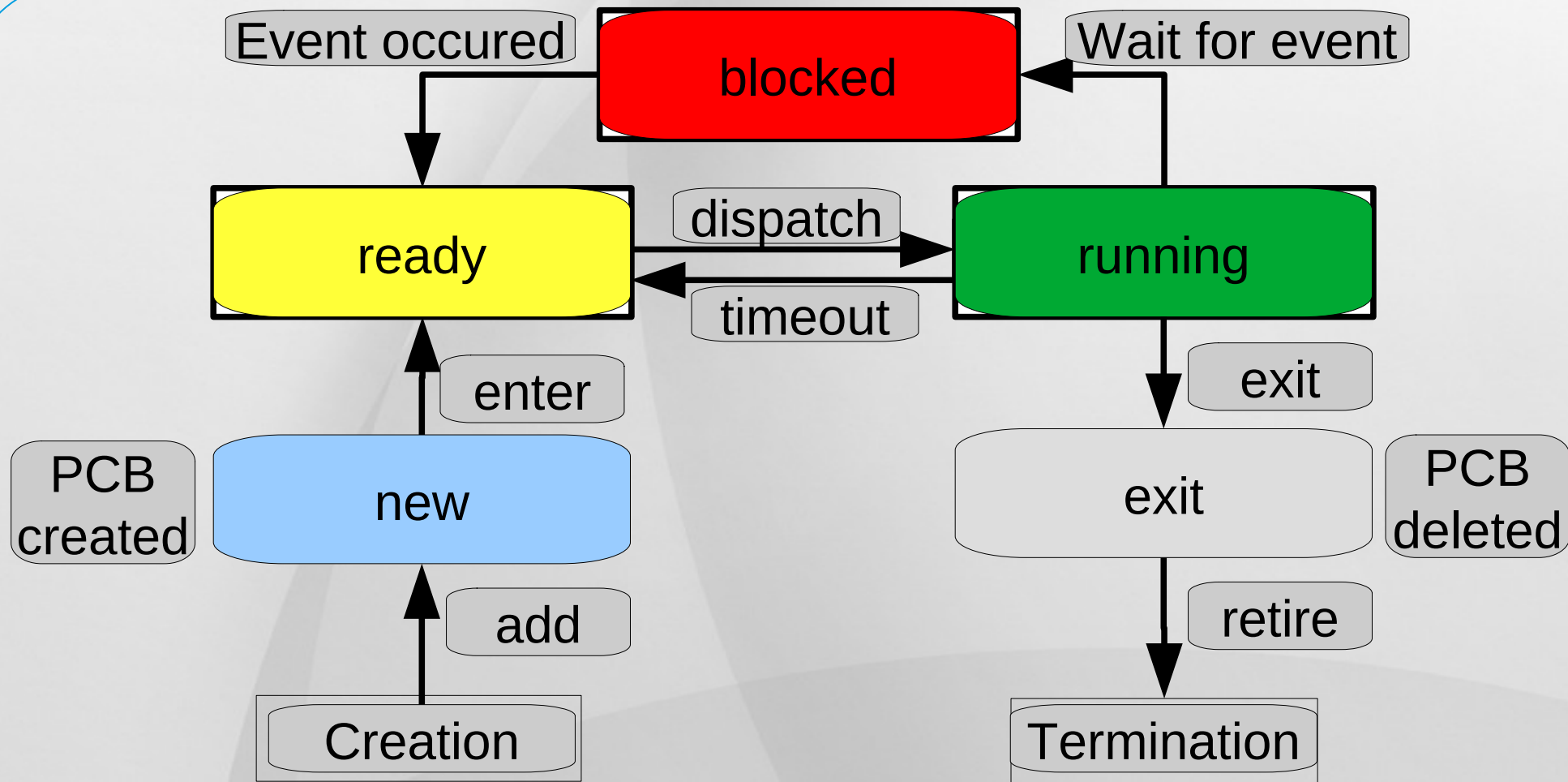
Added 2 states

- **New:** Process has just created (PCB created), but not yet admitted to pool of executable processes
- **Exit:** Process released from pool of executable processes, either halted or aborted for some reason (PCB still exist, ready for deleting)

5-State Model



TH Aschaffenburg
university of applied sciences



Expansion to 6 State Process Model

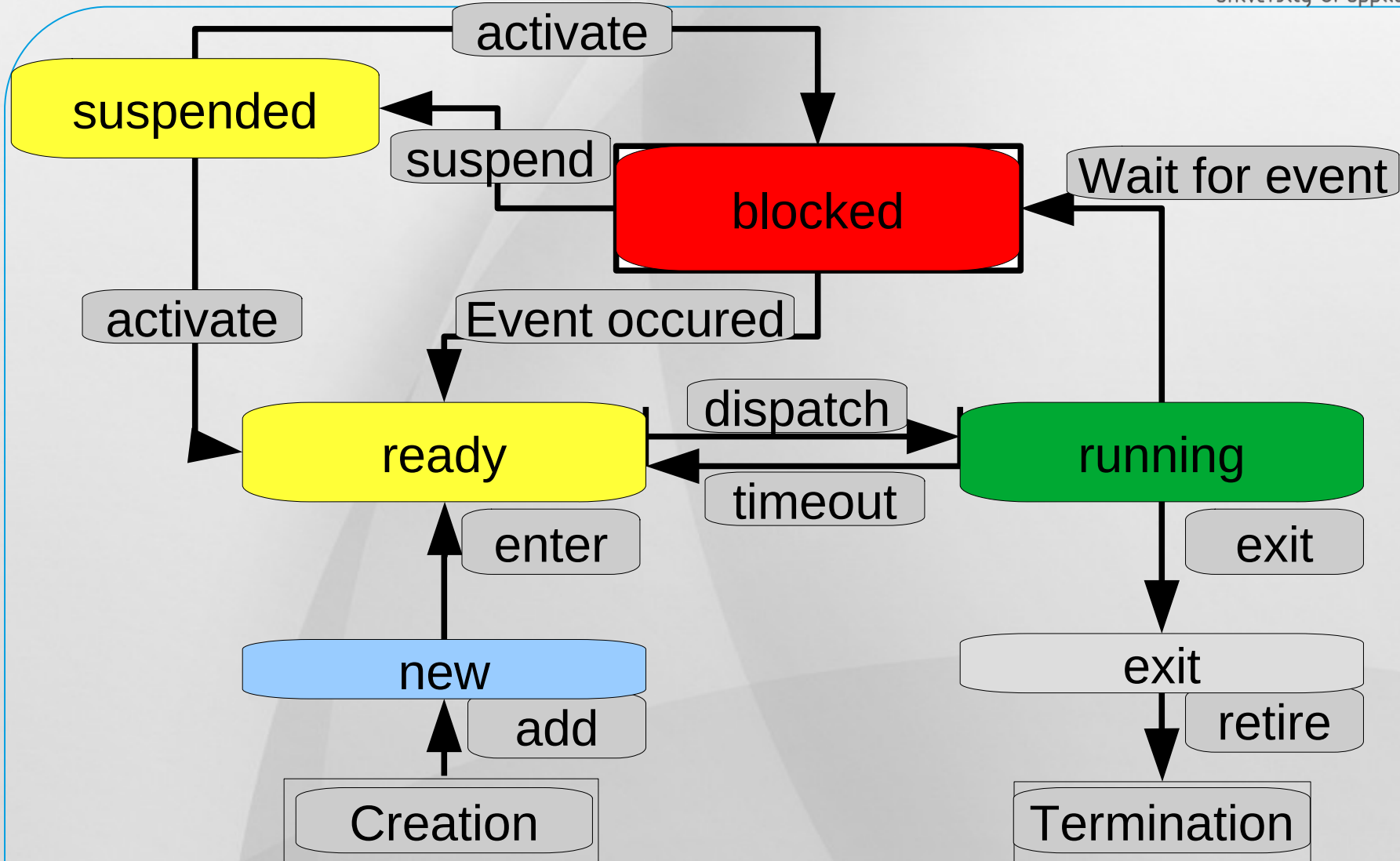
Reasons:

- Main memory has (had very) limited size → additional memory on mass storage (swap space), „secondary memory“

Added 1 suspend state

- **Suspended:**
 - Process, which was blocked before,
 - Process is in secondary memory,
 - Process is still awaiting an activation event

6-State Model



Expansion to 7 State Process Model

Reasons:

- Splitting one suspend state into Ready/Suspend and Blocked/Suspend

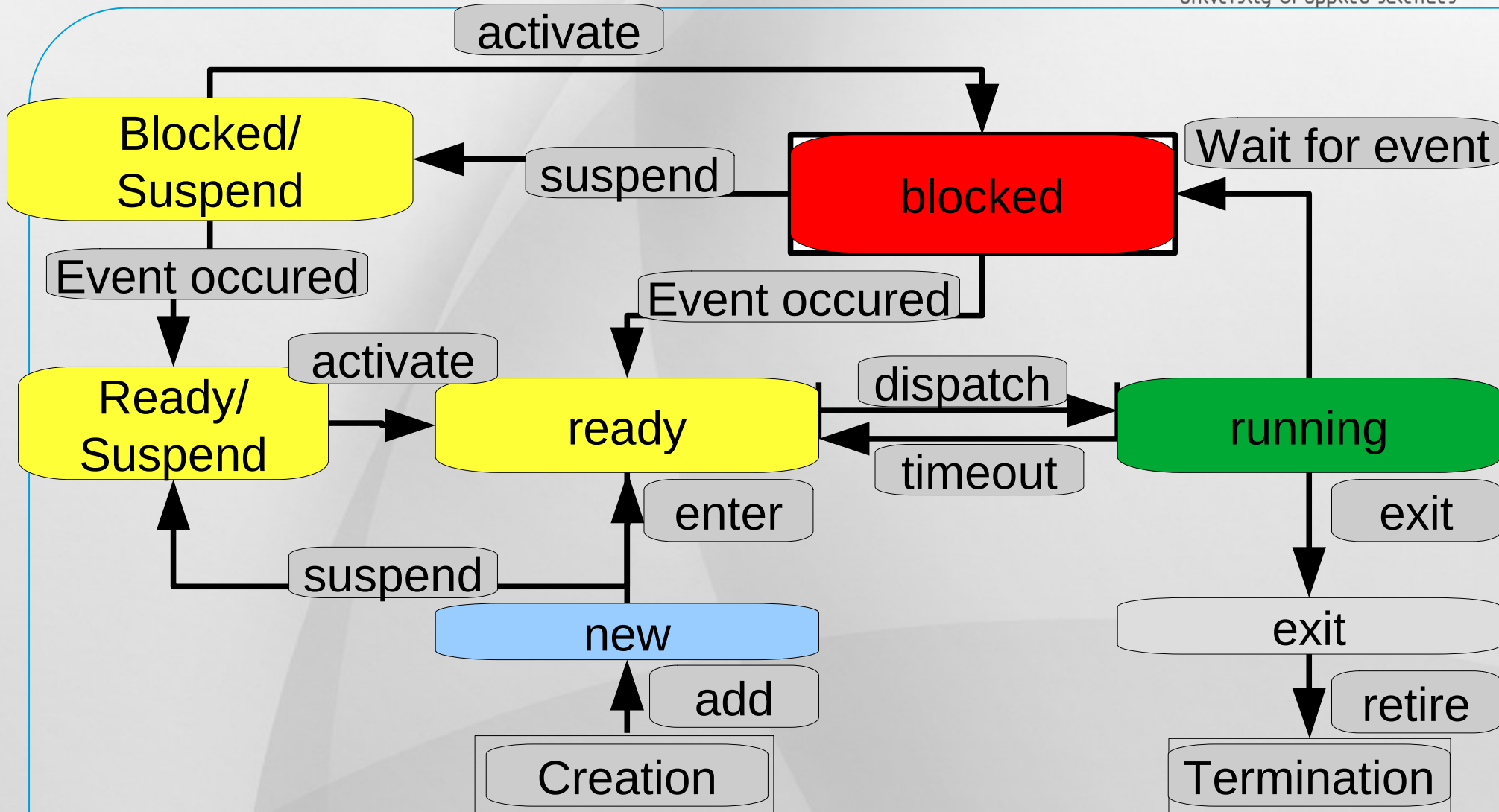
2 suspend states

- **Blocked/Suspended:** Process is blocked while waiting for an event + it is in secondary memory
- **Ready/Suspend:** Process is ready to run, but it is in secondary memory

Advantage:

- More free main memory than in 6 State Model

7-State Model



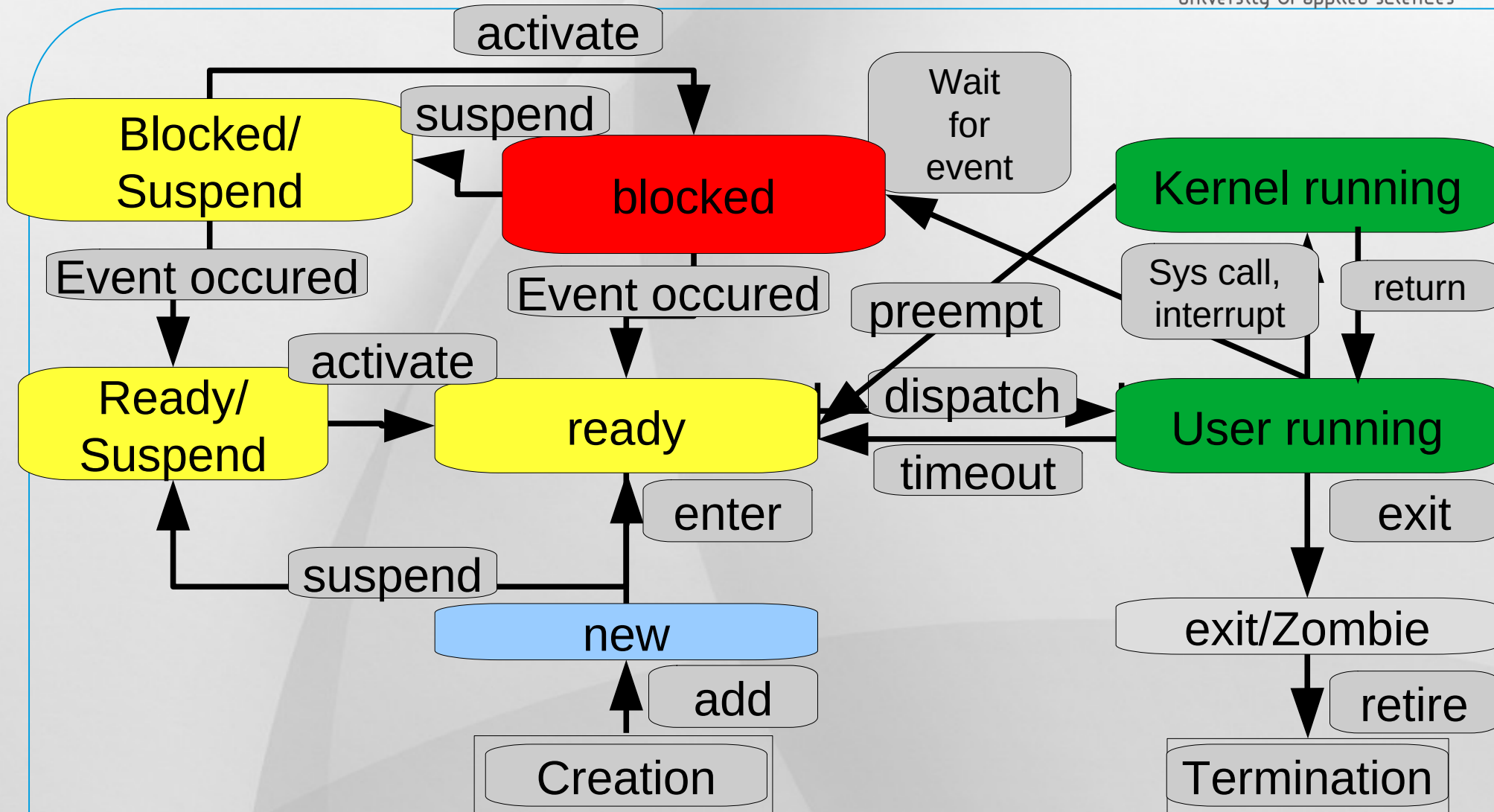
Reasons:

- Splitting up state running into states kernel running and user running
- Different privilege levels

2 Running states

- **Kernel running:** Kernel process runs in kernel mode
- **User running:** Process runs in user mode
- This model is a simplified Unix/Linux State Model (,which contains an explicit state Preempted from kernel running)
- Each model has drawbacks and advantages, selection of model needed while designing an Operating System

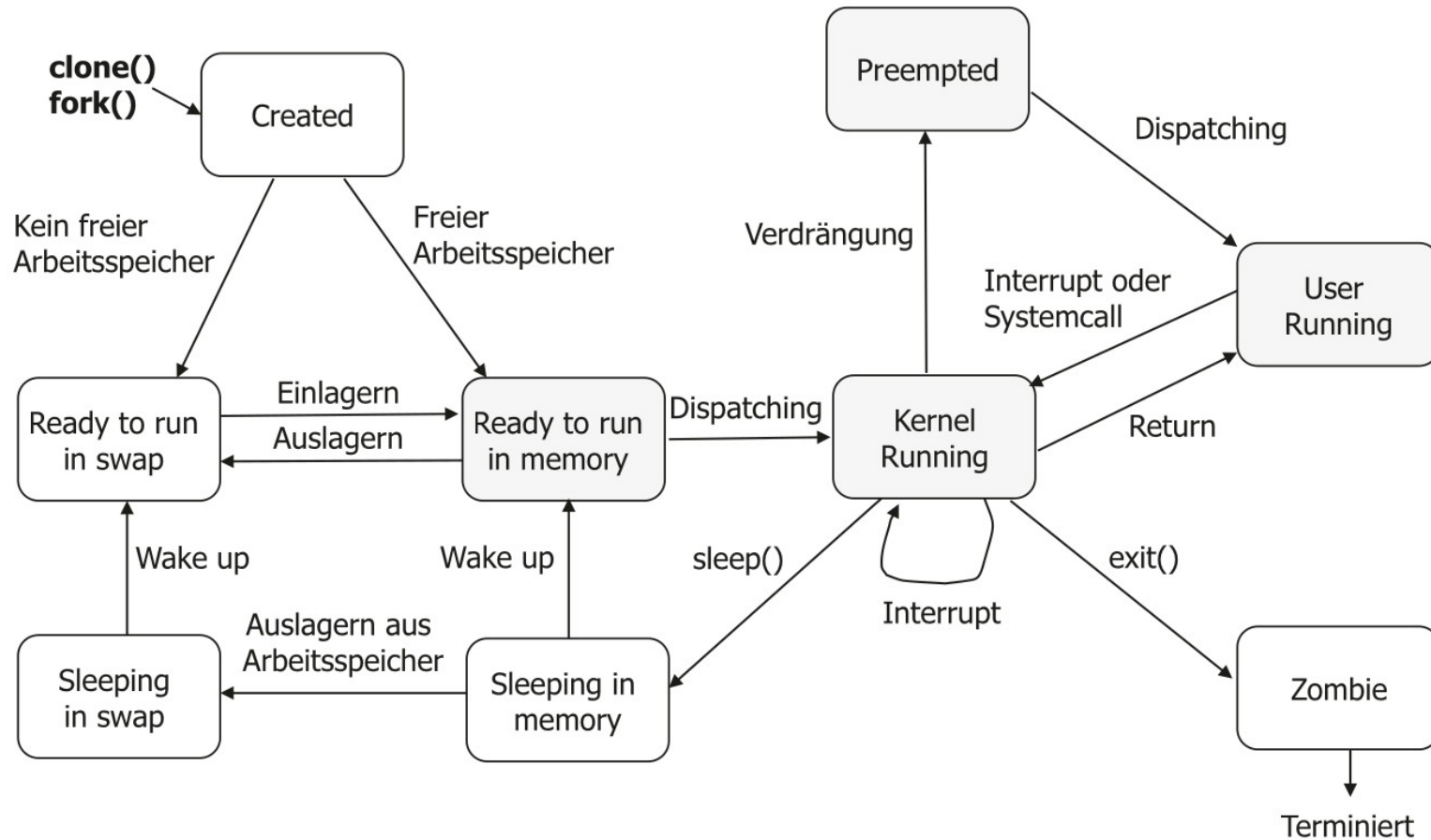
8-State Model



Linux Process Model (in German)



TH Aschaffenburg
university of applied sciences

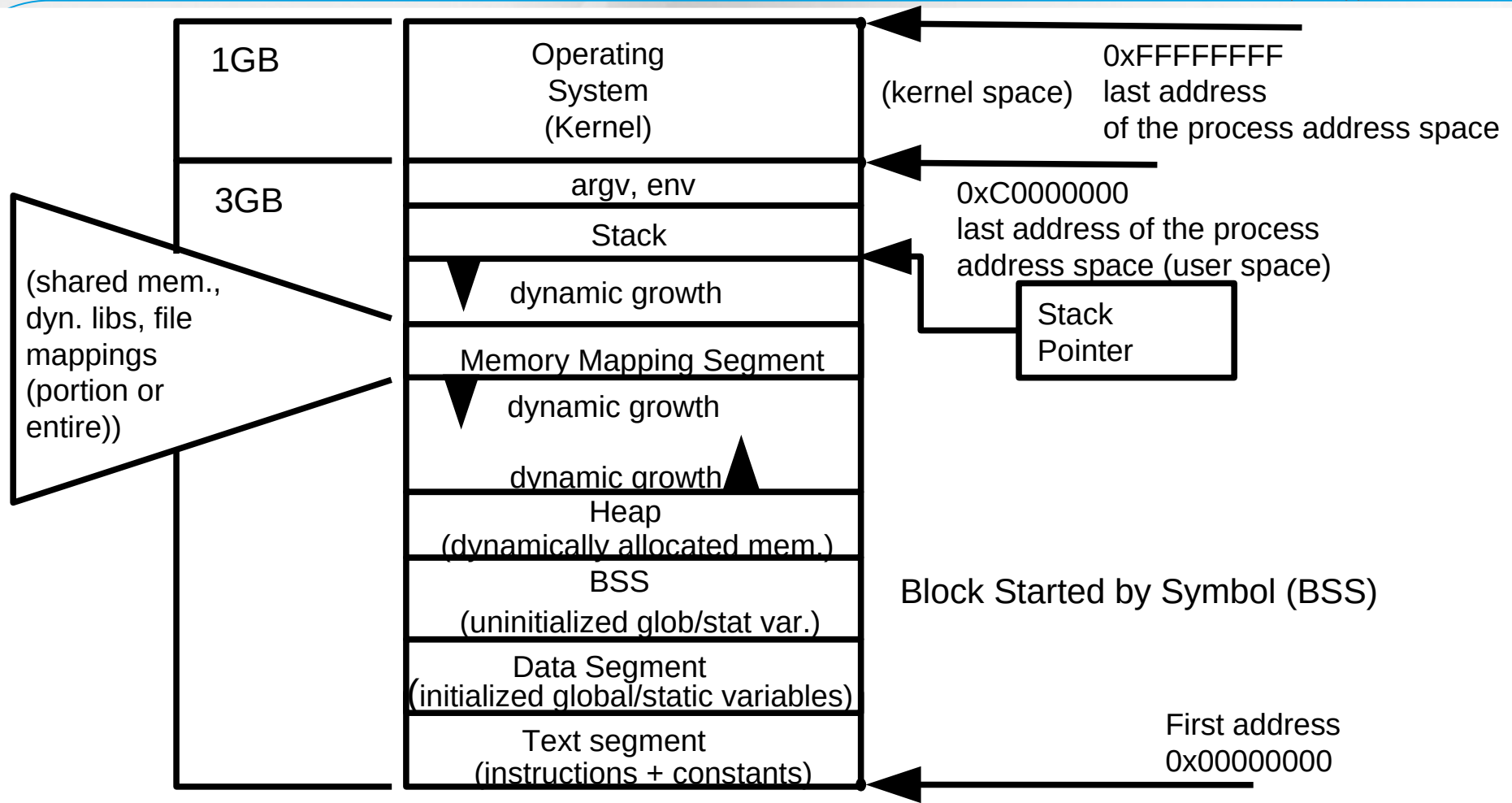


Source: Peter Mandl, Betriebssysteme, 5. Auflage, Springer Vieweg, Wiesbaden 2020
<https://doi.org/10.1007/978-3-658-30547-5>

Process Structure in Memory (e.g. 32 bit Linux, with 4GB mem *sandbox*)



TH Aschaffenburg
university of applied sciences



Source: C. Baun, Operating Systems – bilingual

See also: <https://unix.stackexchange.com/questions/466443/do-memory-mapping-segment-and-heap-grow-until-they-meet-each-other>

Process Structure in Memory - Explanations

- **argv, env** – arguments and environment of the process
- **Stack** – LIFO for subroutine/function calls, parameters, local variables (grows during process runtime)
- **Memory Mapping Segment** – Shared Memory by mmap system call (→ later topic Inter-Process Communication), library and file mappings (grows during process runtime)
- **Heap** – dynamically allocated memory by malloc/free (grows during process runtime)
- **Block Started by Symbol (BSS)** – *historical term*, **uninitialized** global and static variables
- **Data Segment** - **initialized** global and static variables
- **Text Segment** – program code instructions and constants



size <program file> – reports on program

Example 8_3_process_in_mem.c



TH Aschaffenburg
university of applied sciences

```
/* global constant */
const char name[] = "8_3_process_in_mem.c";
/* global variable, initialized */
int global_var_init = 0;
/* global variable, uninitialized */
int global_var_not_init;

int main(int argc, char *argv[])
{
    /* local variable, initialized */
    int local_var_init = 42;
    unsigned int number_of_char = 0;
    char *text = NULL;

    /* local variable, uninitialized */
    int local_var_not_init;
    char input;

    if (argc >= 1)
    {
        printf("argc: %d\n",argc);
        if (argv[0] != NULL)
        {
            printf("argv[0]: %s",argv[0]);
        }
    }
}
```

```
printf("name: %s\n", name);
printf("global_var_init: %d\n",global_var_init);
printf("global_var_not_init: %d\n",global_var_not_init);

printf("local_var_init: %d\n",local_var_init);
printf("local_var_not_init: %d\n",local_var_not_init);

text = malloc(1 * sizeof(char));

if (NULL == text)
{
    fprintf(stderr,"No memory left on heap");
    exit(EXIT_FAILURE);
}
number_of_char = number_of_char + 1;
*text = '\0'; // terminate string correctly

printf("Please input characters: ");
do
{
    scanf("%c",&input);
    text = realloc(text, (number_of_char + 1) * sizeof(char));
    if (NULL == text)
    {
        fprintf(stderr,"No memory left on heap");
        exit(EXIT_FAILURE);
    }
    *(text + number_of_char) = *(text + number_of_char - 1); // move terminating char one
    character
    *(text + number_of_char - 1) = input; // save latest input char
    number_of_char = number_of_char + 1;
} while ('\n' != input);
printf("\n\n");
printf("text: %s\n",text);

return(EXIT_SUCCESS)
```

Example for BSS, Data Segment, Text Segment



```
abke@LT-IW-24-106:~$ size ./Debug/8_3_process_in_mem
```

text	data	bss	dec	hex	filename
2906	656	24	3586	e02	./Debug/8_3_process_in_mem



text+data+bss in decimal and hexadecimal